# LexVault AI - Capstone Project Review Guide

## 1. One-Minute Project Introduction

- **Project name:** LexVault AI.
- **Project category:** Secure institutional legal-knowledge platform for Indian law firms.
- **Core problem:** Legal firms lose senior-lawyer experience because it remains in personal notes, voice memos, and case files instead of a reusable institutional knowledge base.
- **Privacy problem:** Raw client information must not be sent to public AI tools because of attorney-client privilege, confidentiality, and professional-ethics requirements.
- **Our solution:** LexVault captures working knowledge, removes personally identifiable information, converts case-specific facts into reusable legal strategy, requires lawyer approval, and makes only approved knowledge searchable.
- **Main differentiator:** Privacy and human review are part of the architecture, not post-processing steps.

## 2. Project Objectives

- Build a working legal knowledge capture workflow.
- Keep client-identifying information out of reusable institutional knowledge.
- Add a mandatory human-in-the-loop approval gate.
- Support firm-level multi-tenancy so one firm cannot retrieve another firm's knowledge.
- Provide semantic search, graph exploration, and a guarded AI assistant.
- Include Indian legal resources such as Constitution articles, statutes, and IPC-to-BNS mappings.
- Provide practical litigation tools for legal teams.
- Make important privacy, approval, and assistant actions auditable.

## 3. Product Capabilities

- **Personal Knowledge Vault:** Capture typed notes, voice input, and uploaded legal documents/images.
- **Secure Knowledge Publishing:** Sanitize and abstract knowledge before publication.
- **Firm Intelligence Hub:** Browse and search approved precedents and institutional insights.
- **Safe AI Assistant:** Answer questions using firm knowledge with source-aware guardrails.
- **Constitution Explorer:** Search constitutional parts, articles, English text, Hindi text, and related precedents.
- **Statute Library:** Browse legal acts, sections, and selected codified statute content.
- **BNS Mapping:** Resolve older IPC, CrPC, and IEA references to newer BNS, BNSS, and BSA references.
- **Limitation Calculator:** Calculate limitation periods for supported matter types.
- **Conflict Checker:** Check internal records for potential conflicts.
- **Judgment Summarizer:** Organize judgment material into legal issues, ratio, and orders.
- **Associate Quiz:** Train junior associates using institutional legal material.
- **Hearing Tracker:** Track upcoming hearings and court diary information.
- **Admin Console:** View users, statistics, audit activity, and tenant-level data.

## 4. Eight-Stage Pipeline Presentation

Use this as the main project demonstration. Explain that every major screen maps to one or more stages.

### Stage 1 - Raw Capture

- A lawyer enters a note, records a voice memo, or uploads an image/PDF.
- The capture page includes Indian legal case templates for a repeatable demo.
- The backend creates a knowledge item and pipeline job.
- The item starts in a processing state and is assigned to the authenticated lawyer's firm.

### Stage 2 - Local AI Processing

- Raw text is cleaned and structured.
- Voice and OCR inputs can be normalized into usable text.
- A title and readable working content are generated.
- The design prioritizes local processing so confidential raw facts do not need to be sent to a public model.

### Stage 3 - PII Removal and Sanitization

- The system detects client names, organizations, locations, emails, phone numbers, and legal identifiers where supported.
- Detected entities are replaced with safe placeholders.
- The system stores both sanitized text and detected-entity audit information.
- The pipeline UI shows what was detected and what was removed.

### Stage 4 - Strategy Abstraction

- The sanitized matter is converted from a specific case narrative into a reusable legal pattern.
- Specific parties and dates become a general strategy for handling a similar legal issue.
- The goal is to preserve legal reasoning while removing unnecessary client identity.

### Stage 5 - Lawyer Approval Gate

- The item enters the approval queue.
- A reviewer or administrator can inspect sanitized text, abstracted insight, and PII findings.
- The reviewer can approve, reject, or edit before approving.
- No item should be published to shared retrieval stores without an explicit approval action.
- Approval identity, timestamp, and action are recorded for auditability.

### Stage 6 - Vector Database and Knowledge Graph

- Approved knowledge is embedded for semantic retrieval.
- Qdrant is used for vector search.
- Neo4j is used for relationships among knowledge items, concepts, practice areas, and lawyers.
- Firm identifiers are carried into storage and retrieval filters.

### Stage 7 - RAG Retrieval

- A lawyer asks a natural-language question in the assistant or knowledge hub.
- The system searches approved institutional knowledge.
- Retrieval can combine semantic similarity with structured legal relationships.
- Results are restricted to the current user's firm.

### Stage 8 - Safe Response

- The assistant generates a response from retrieved internal material.
- The response is designed to identify the internal knowledge sources used.

- The assistant should not invent a firm precedent when retrieval has no supporting result.
- Assistant queries are recorded in the audit/history layer.

## 5. Architecture Overview

- **Frontend:** Next.js 14 App Router, TypeScript, Tailwind CSS, React Flow, Framer Motion, Lucide icons, Zustand, and Socket.IO client.
- **Backend:** FastAPI with asynchronous SQLModel/SQLAlchemy database access.
- **Relational database:** PostgreSQL for users, firms, knowledge items, pipeline jobs, approvals, and audit records.
- **Vector database:** Qdrant for semantic knowledge retrieval.
- **Graph database:** Neo4j for concepts, practice areas, authors, and knowledge relationships.
- **Local model service:** Ollama for local model execution.
- **Privacy services:** Presidio-compatible PII detection/anonymization path and resilient fallback analyzer.
- **Transport:** REST APIs under `/api` plus Socket.IO updates under `/ws`.
- **Authentication:** JWT-based login with role and firm information.
- **Database fallback:** SQLite fallback is configured for environments where PostgreSQL is unavailable.

## 6. Important Repository Areas

- `frontend/app/capture/page.tsx`: capture, voice, OCR, language selection, and case templates.
- `frontend/app/pipeline/[jobId]/page.tsx`: live pipeline monitor, WebSocket status, polling fallback, and stage output inspection.
- `frontend/app/approvals/page.tsx`: reviewer approval workflow.
- `frontend/app/knowledge-hub/page.tsx`: institutional knowledge browsing and search.
- `frontend/app/assistant/page.tsx`: guarded assistant experience.
- `frontend/app/graph/page.tsx`: graph visualization.
- `frontend/app/tools/page.tsx`: legal tools and calculators.
- `frontend/lib/api.ts`: frontend API access functions.
- `backend/app/main.py`: FastAPI application, CORS, lifespan, routes, and WebSocket mount.
- `backend/app/pipeline/runner.py`: orchestration of capture stages and approval/storage stages.
- `backend/app/pipeline/stage1_capture.py` through `stage8_response.py`: individual pipeline responsibilities.
- `backend/app/models/`: relational data models.
- `backend/app/api/`: authentication, capture, pipeline, approval, search, chat, graph, admin, constitution, statute, and tool routes.
- `backend/app/services/`: embedding, OCR, LLM, PII, Qdrant, Neo4j, and voice integrations.
- `backend/app/data/`: Indian Constitution, statute, and BNS/IPC reference data.

## 7. Suggested Live Demonstration

1. Open `http://localhost:3000` and introduce the problem and eight-stage diagram.
2. Sign in with a demo lawyer account from the README.
3. Open Capture and select a pre-loaded Indian legal case template.
4. Submit the capture and show the live pipeline job page.
5. Explain the cleaned text and PII-sanitization outputs.
6. Open Approvals and show the human review gate.

7. Approve the sanitized insight as a reviewer/admin.
8. Open Knowledge Hub and show the approved item.
9. Ask the Assistant a question related to an approved institutional precedent.
10. Open Graph to show connected concepts and practice areas.
11. Open Constitution, Statutes, or Tools to demonstrate legal-domain breadth.
12. Open Admin and show audit activity and tenant-aware statistics.
13. Briefly demonstrate the second firm account and explain that its knowledge is isolated.

## 8. Demo Credentials

- **Chandra & Rao Associates:** `rohit@chandrarao.law` / `password123`
- **Chandra & Rao senior partner/admin:** `meera.rao@chandrarao.law` / `password123`
- **NyayaVault India:** `arjun@nyayavault.in` / `password123`
- **NyayaVault managing partner/admin:** `priya@nyayavault.in` / `password123`

Use demo credentials only in the local capstone environment. Do not use them for a deployed system.

## 9. How to Run the Project on Windows

### Prerequisites

- Docker Desktop with Docker Compose.
- Python 3.11 or newer.
- Node.js 18 or newer and npm.

### Start infrastructure

```
Set-Location D:\capstone
docker compose up -d
```

This starts PostgreSQL on `5432`, Qdrant on `6333`, Neo4j Browser on `7474`, Neo4j Bolt on `7687`, and Ollama on `11434`.

### Start backend

```
Set-Location D:\capstone\backend
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
python -m app.seed
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Backend URLs:

- Health: `http://localhost:8000/health`
- Root: `http://localhost:8000/`
- Swagger API documentation: `http://localhost:8000/docs`

### Start frontend

```
Set-Location D:\capstone\frontend
npm install
npm run dev
```

Frontend URL: `http://localhost:3000`

### Optional combined command

```
Set-Location D:\capstone
make dev
```

On Windows, running the backend and frontend in separate terminals is usually easier to inspect.

## 10. Configuration Points

- Frontend API URL: `NEXT_PUBLIC_API_URL=http://localhost:8000`.
- Frontend WebSocket URL: `NEXT_PUBLIC_WS_URL=http://localhost:8000`.
- PostgreSQL connection: `DATABASE_URL`.
- Qdrant connection: `QDRANT_URL`.
- Neo4j connection and credentials: `NEO4J_URI`, `NEO4J_USER`, `NEO4J_PASSWORD`.
- Ollama connection and model: `OLLAMA_URL`, `OLLAMA_MODEL`.
- Optional Groq model: `GROQ_API_KEY`, `GROQ_MODEL`.
- JWT configuration: `JWT_SECRET`, `JWT_ALGORITHM`, and token expiry.
- Use `.env.example` as the configuration reference and keep real secrets out of source control.

## 11. Validation Evidence

- Docker Compose infrastructure started successfully with PostgreSQL, Qdrant, Neo4j, and Ollama containers running.
- Backend started successfully on port `8000`.
- `GET /health` returned HTTP `200` with `{"status":"healthy","service":"lexvault-backend"}`.
- `GET /` returned HTTP `200` and reported LexVault AI as online.
- Frontend development server started successfully on port `3000`.
- Frontend root page returned HTTP `200`.
- Frontend production build passed compilation, lint/type checking, page generation, and route generation.
- The multi-tenancy verification script passed:
- Firm 1 contained 5 knowledge items.
- Firm 2 contained 10 knowledge items.
- Overlapping knowledge IDs were an empty set.
- Result: strict multi-tenancy isolation verified.

## 12. Current Environment Note for Honest Review Communication

- The application starts successfully with resilient fallbacks when optional Python integrations are not installed in the current interpreter.
- The current startup log reported fallback mode for the full Presidio engine, SentenceTransformer embeddings, the Qdrant Python client, the Neo4j Python driver, and faster-whisper.
- The Docker services themselves were running, but the corresponding Python client packages were not available to the active Python environment.
- For the strongest final demonstration, install `backend/requirements.txt` inside the active virtual environment and verify model/data assets are available.
- Present the architecture as the intended production path, and distinguish it from fallback mode when discussing this local run.
- Do not claim that a missing integration was exercised live. Show the startup logs or dependency installation status if reviewers ask.

## 13. Team Presentation Split

- **Member 1 - Problem and users:** Explain the institutional-memory problem, attorney-client privilege risk, target law-firm users, and success criteria.

- **Member 2 - Architecture:** Explain Next.js, FastAPI, PostgreSQL, Qdrant, Neo4j, Ollama, JWT, REST, and WebSocket responsibilities.
- **Member 3 - Privacy pipeline:** Demonstrate capture, local processing, PII removal, strategy abstraction, and explain why raw facts stop before publication.
- **Member 4 - Human review and retrieval:** Demonstrate approval, publication, Knowledge Hub, graph view, assistant retrieval, and guardrails.
- **Member 5 - Testing and impact:** Demonstrate tenant-isolation evidence, API health, production build, legal resources, limitations, and future work.

## 14. Likely Review Questions and Answers

### Why not send all data directly to ChatGPT?

- Raw client data may contain privileged or confidential information.
- LexVault sanitizes and abstracts the content before it becomes reusable knowledge.
- The architecture favors local processing and includes a human approval gate.

### What prevents Firm A from seeing Firm B data?

- Users carry a `firm_id`.
- Knowledge items are stored with a `firm_id`.
- Retrieval and storage layers are designed to apply firm-level filtering.
- The repository's isolation test found no overlapping knowledge IDs between the two seeded firms.

### Why is human approval necessary?

- PII detection and AI abstraction are not infallible.
- A senior lawyer must decide whether an insight is accurate, safe, and suitable for firm-wide reuse.
- Approval identity and timestamp create accountability.

### What happens if no precedent is found?

- The safe assistant should avoid presenting an unsupported institutional answer.
- This is preferable to confident hallucination in a legal workflow.

### Why use both a vector database and a graph database?

- Vectors are useful for semantic similarity and natural-language search.
- Graphs represent explicit relationships between concepts, practice areas, lawyers, and knowledge items.
- Together they support broader and more explainable retrieval than either structure alone.

### Is this a legal advice system?

- No. It is an internal knowledge-management and research support tool.
- A qualified lawyer remains responsible for legal judgment, verification, and client advice.

### What are the current limitations?

- Model and external integration quality depends on local dependencies, model downloads, and service availability.
- The prototype is not a substitute for a production security review, formal legal compliance assessment, or court-authoritative legal research database.
- OCR, multilingual processing, and legal citation coverage should be expanded and benchmarked with expert-reviewed datasets.

## 15. Future Enhancements

- Add automated integration tests for every pipeline stage.
- Add stronger custom Indian identifiers for PII detection, including case numbers and additional document formats.
- Add stronger role-based permissions beyond the current lawyer/reviewer/admin model.
- Add immutable, exportable audit reports.
- Add retrieval evaluation sets for precision, recall, grounding, and tenant isolation.
- Add production secrets management and encrypted storage.
- Add background job workers for long-running OCR, transcription, and embedding tasks.
- Add court-source verification and legal-content update workflows.
- Add Hindi and regional-language model evaluation with legal-domain reviewers.

## 16. Closing Statement

LexVault AI demonstrates how a legal technology platform can preserve institutional expertise without treating privacy as an afterthought. The central design decision is the controlled path from raw capture to sanitized, abstracted, lawyer-approved knowledge. That path is visible in the product, testable in the backend, and explainable during the project review.